Spring Boot 3 + JWT Security (Production-Grade)

Overview

This guide provides a production-ready implementation of JWT authentication using Spring Boot 3 and Spring Security 6.

Dependencies (pom.xml)

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.11.5</version>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.11.5</version>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.11.5</version>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

---

application.yml

```
jwt:
  secret: mySuperSecretKey1234567890
```

```
  expiration: 3600000
```

JwtService.java

```
@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secret;

    public String generateToken(String username) {
        return Jwts.builder()
                .subject(username)
                .issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + 3600000))
                .signWith(getKey())
                .compact();
    }

    public String extractUsername(String token) {
        return extractClaims(token).getSubject();
    }

    private Claims extractClaims(String token) {
        return Jwts.parser()
                .verifyWith(getKey())
                .build()
                .parseSignedClaims(token)
                .getPayload();
    }

    private SecretKey getKey() {
        return Keys.hmacShaKeyFor(secret.getBytes());
    }
}
```

SecurityConfig.java

```
@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtFilter jwtFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
```

```
        return http
                .csrf(csrf -> csrf.disable())
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/auth/**").permitAll()
                        .anyRequest().authenticated()
                )
                .sessionManagement(sess ->
sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class)
                .build();
    }
}
```

JwtFilter.java

```
@Component
@RequiredArgsConstructor
public class JwtFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);
        String username = jwtService.extractUsername(token);

        if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
            UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(username, null,
List.of());

            SecurityContextHolder.getContext().setAuthentication(authToken);
        }

        filterChain.doFilter(request, response);
    }
}
```

AuthController.java

```
@RestController
@RequestMapping("/auth")
@RequiredArgsConstructor
public class AuthController {

    private final JwtService jwtService;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody Map<String, String> request) {

        if ("admin".equals(request.get("username")) &&
            "password".equals(request.get("password"))) {

            String token = jwtService.generateToken(request.get("username"));
            return ResponseEntity.ok(Map.of("token", token));
        }

        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
    }
}
```

Key Production Features

- Stateless authentication
- Secure filter chain
- Token validation on every request
- Externalized configuration
- Spring Security 6 compatible

Next Improvements

- Add refresh tokens
- Use database-backed users
- Add roles & authorities
- Use BCrypt password encoder